Project Report: Containerized CI/CD Pipeline on ECS Fargate

Mapped to AWS Certified Developer – Associate (DVA-C02) Exam Domains

Project: Node.js app → Docker → ECR → ECS Fargate → CodePipeline (rolling deploy)
Purpose of this report: connect what you *actually built and debugged* to the specific skills DVA-C02 tests, so the exam questions read as "oh, I hit this" instead of abstract trivia.

---

1. Project Summary

You containerized a Node.js/Express app, pushed it to Amazon ECR, deployed it to ECS on Fargate with a task definition, and automated the whole path with CodePipeline (Source → Build → Deploy) so that a `git push` triggers a build and a zero-downtime rolling deployment.

You built this **two ways** — manually via AWS CLI (Option B), and as Infrastructure-as-Code via CloudFormation (Option A) — which is exactly the kind of dual exposure DVA-C02 rewards, since the exam tests both "how do you do X with the CLI/SDK" and "how do you express X as IaC."

---

2. DVA-C02 Domain Weighting (know this cold)

Domain	Weight	What it covers
1. Development with AWS Services	32%	Writing code that uses AWS APIs/SDKs, containers, serverless
2. Security	26%	IAM, authN/authZ, encryption, credential management
3. Deployment	24%	CI/CD, IaC, deployment strategies
4. Troubleshooting and Optimization	18%	Logging, monitoring, root-cause analysis, performance

Every AWS CLI error you hit in this project maps to one of these four buckets. That mapping is the rest of this report.

---

3. Domain 1 — Development (32%)

3.1 Containerization concepts

- **`Dockerfile` layering**: your build cached `npm install` separately from `COPY app/`, so code changes didn't force a full dependency reinstall. Exam angle: understand why layer order matters for build speed, and that CodeBuild's `PrivilegedMode: true` is required specifically to run `docker build` inside a CodeBuild container (Docker-in-Docker).
- **Multi-stage tagging**: your `buildspec.yml` tagged images two ways — the git commit hash (immutable, traceable) and `latest` (mutable, convenience). Exam angle: DVA-C02 tests whether you know ECR supports **immutable tag** repository settings, and why commit-hash tagging is the production-safe pattern (rollback by exact tag, not by "whatever `latest` happened to be").

3.2 ECS task definitions

- **`containerDefinitions`**, **`portMappings`**, **`logConfiguration`** — you hand-wrote all of these. Exam angle: know the difference between **task-level** settings (`cpu`, `memory`, `networkMode`, `requiresCompatibilities`) and **container-level** settings (per-container `cpu`/`memory` limits, health checks). Expect a question distinguishing Fargate's mandatory `awsvpc` network mode from EC2 launch type's optional `bridge`/`host` modes.
- **Health checks**: you added a container-level `HEALTHCHECK` in the Dockerfile *and* an ECS-level `healthCheck` block in the task def. Exam angle: these are two different mechanisms — Docker's own healthcheck vs. ECS's, and (separately) an ALB target group health check if one were in front. Know which layer reports what.

3.3 SDK/API usage pattern

Your `buildspec.yml` scripted `aws ecr get-login-password` piped into `docker login` — this is the canonical way services authenticate to ECR programmatically. Exam angle: DVA-C02 loves testing "how does a CI job authenticate to ECR" — the answer is always a short-lived token via `GetAuthorizationToken`, never long-lived static credentials baked into the image.

---

4. Domain 2 — Security (26%)

This domain is where your project generated the most exam-relevant friction, and friction is what teaches this domain.

4.1 Task Role vs. Task Execution Role — the #1 DVA-C02 trap

You built an `ecsTaskExecutionRole` with the AWS-managed

`AmazonECSTaskExecutionRolePolicy`. This role's *only* job is letting the **ECS agent** pull the image from ECR and write logs to CloudWatch — it has nothing to do with what your *application code* is allowed to do at runtime (e.g., calling S3 or DynamoDB from inside the container). That's a separate, optional **Task Role**. The exam will absolutely test this distinction with a scenario like "the container can't read from an S3 bucket" — the fix is a Task Role with S3 permissions, not touching the Execution Role.

4.2 IAM least-privilege, scoped by ARN

Your CloudFormation's `CodeBuildServiceRole` scoped ECR push permissions to `!GetAtt ECRRepository.Arn` specifically, rather than `Resource: "*"`. Exam angle: know that `ecr:GetAuthorizationToken` is a special case — it must **always** be `Resource: "*"` (there's no per-repo variant of that specific action) while `ecr:PutImage`/`BatchGetImage`/etc. **can** and should be scoped to specific repo ARNs.

4.3 `iam:PassRole` — a recurring exam gotcha

Your `CodePipelineServiceRole` needed explicit `iam:PassRole` permission for the Execution Role before CodePipeline could hand it off to the ECS deploy action. Exam angle: **any time one AWS service must assign an IAM role to a resource on your behalf** (CodePipeline→ECS, Lambda execution role assignment, EC2 instance profile attachment), the calling principal needs `iam:PassRole` on that specific role ARN — a very common "why is this AccessDenied" scenario on the exam.

4.4 Encryption at rest

Your `ArtifactBucket` set `ServerSideEncryptionByDefault: AES256` (SSE-S3). Exam angle: know the tradeoff vs. SSE-KMS (customer-managed keys, audit trail via CloudTrail, extra cost/latency) — expect at least one question asking you to pick the right encryption mode for a stated compliance requirement.

---

5. Domain 3 — Deployment (24%)

This is the heart of the project, and where the real debugging happened.

5.1 Deployment controller types

Your ECS Service used `DeploymentController: { Type: ECS }` — the native **rolling update** controller, governed by `MinimumHealthyPercent` (50) and `MaximumPercent` (200). Exam angle: DVA-C02 tests all three ECS deployment controller types:

- **ECS (rolling update)** — what you built; gradual replace, governed by the two percentage knobs
- **CODE_DEPLOY (blue/green)** — needs a second target group + `appspec.yaml`, enables traffic shifting and easy rollback
- **EXTERNAL** — you manage task placement yourself

Know when to pick blue/green over rolling: when you need **instant rollback** or **canary traffic shifting** — rolling update can't do either cleanly.

5.2 Deployment Circuit Breaker — added mid-project, directly from a real failure

You hit a real deployment that hung for 10+ minutes with no clear failure signal. The fix was adding:

```yaml
DeploymentCircuitBreaker:
  Enable: true
  Rollback: true
```

Exam angle: this is a **named, testable feature**. Without it, a broken deployment (bad image, failing health check) can retry indefinitely, consuming time and, on EC2 launch type, capacity. With it, ECS detects repeated task failures and auto-rolls-back to the last known-good task definition. Expect a scenario question: "deployments hang without clear errors — what feature should be enabled?"

5.3 CodePipeline stage anatomy

Source → Build → Deploy, each stage with `InputArtifacts`/`OutputArtifacts` chained through S3. Exam angle: know that **artifacts pass between stages via the S3 artifact bucket**, not directly between compute environments — this is why your `ArtifactBucket` needed both CodeBuild's and CodePipeline's roles to have `s3:GetObject`/`PutObject` on it.

5.4 `imagedefinitions.json` — the ECS deploy action's contract

Your `buildspec.yml`'s `post_build` phase generated this file:

```json
[{"name":"ecs-demo-app","imageUri":"<repo>:<tag>"}]
```

Exam angle: this is the **exact, required format** for CodePipeline's native ECS deploy action — the `name` field must match the `containerName` in your task definition exactly, or the deploy action fails silently-ish with a mapping error. This is a very specific, testable detail.

5.5 CloudFormation change sets and stack states — learned entirely from your real errors

You hit, in order:

- `RepositoryAlreadyExistsException` — a resource with that name existed outside the stack's ownership
- `AWS::EarlyValidation::ResourceExistenceCheck` failure — CFN's pre-flight check blocking changeset creation because of a **naming collision** with manually-created

resources
- `REVIEW_IN_PROGRESS` stuck state — a changeset created but never executed
- `ROLLBACK_COMPLETE` — a failed create, where CFN **cannot** retry in place; it must be deleted and recreated

Exam angle: DVA-C02 tests CloudFormation stack **state machine** knowledge directly — which states allow update-in-place vs. require delete/recreate, what `--no-fail-on-empty-changeset` does, and that changesets are a *preview* step, separate from execution.

---

6. Domain 4 — Troubleshooting and Optimization (18%)

This is arguably the domain your actual session generated the most raw material for.

6.1 The `docker push :latest` corruption

Root cause: a dropped `:` character during copy/paste turned `ecs-demo-app:latest` into `ecs-demo-appatest`, a nonexistent repo name. Exam angle: not itself an exam topic, but it reinforced **reading error messages literally** — "repository does not exist" was 100% accurate; the repository name was simply wrong, not "ECR broken."

6.2 `ecs register-task-definition --role ...` invalid flag

Root cause: assuming a CLI flag exists that doesn't — the Task Execution Role is a *field inside the JSON payload* (`executionRoleArn`), not a CLI parameter. Exam angle: DVA-C02 expects you to know **which settings live in the task definition JSON vs. which are CLI/API call parameters** — a recurring theme across ECS, Lambda, and CodeDeploy CLI questions.

6.3 The ECS Service stuck 10+ minutes (before the circuit breaker fix)

Root cause candidates you correctly diagnosed toward: subnets lacking a route to an Internet Gateway, meaning Fargate tasks (which need outbound internet to pull from ECR and ship logs to CloudWatch, absent VPC endpoints) never reach `RUNNING`. Exam angle: this is a **classic DVA-C02 troubleshooting scenario** — "Fargate tasks stuck in PENDING/PROVISIONING" almost always traces to one of:

- No route to IGW (public subnet misconfigured) or no NAT Gateway (private subnet with no egress)
- Missing VPC endpoints for ECR/S3/CloudWatch Logs if deliberately running with no internet egress at all
- Security group blocking egress (rare, since default SGs allow all outbound, but worth knowing it's checkable)

6.4 Root-cause tools you used, which the exam explicitly tests

- `aws cloudformation describe-stack-events` — filtering for `FAILED` resources to

find the *first* failure in a cascade (later failures are often just downstream noise from the first one)

- `aws ecs describe-tasks --query "tasks[].stoppedReason"` — the single most useful field for diagnosing why a Fargate task died
- `aws logs tail --follow` — CloudWatch Logs Insights / `tail` as the first stop for application-level (not infrastructure-level) failures
- `aws ecr describe-images` — confirming image tags actually landed, rather than assuming a push succeeded

---

7. Exam-Style Self-Check Questions (write your own answers before checking)

1. A CodePipeline ECS deploy action fails with an error about the image definitions file. What are the two most likely causes? *(Hint: filename mismatch with pipeline config, or `name` field not matching container name.)*
2. Your ECS service's tasks keep cycling between `PROVISIONING` and `STOPPED`, and the circuit breaker eventually rolls back the deployment. Name three distinct root causes to check first, in order of likelihood.
3. Your CodeBuild project's IAM role has `ecr:PutImage` scoped to one repo ARN, but the build still fails to authenticate to ECR. What's missing, and why can't it be scoped the same way?
4. You need your Fargate container's application code to read from a DynamoDB table. Which IAM role do you attach the DynamoDB permissions to — the Task Role or the Task Execution Role — and why is this a graded distinction on the exam?
5. A CloudFormation stack is in `ROLLBACK_COMPLETE`. You update the template to fix the bug and run `aws cloudformation deploy` again. What happens, and why?

(Answers: 1—filename must match pipeline config exactly, and the `name` key inside imagedefinitions.json must exactly match the container name in the task def. 2—no IGW/NAT route for outbound internet, bad image/tag, or failing container health check. 3—`ecr:GetAuthorizationToken` must be `Resource: "*"` regardless of other scoped permissions, since it's account-level, not repo-level. 4—Task Role; the Execution Role is for the ECS agent's own actions (pulling image, writing logs), never for your application's runtime AWS API calls. 5—the deploy fails immediately; CFN refuses to update a stack sitting in `ROLLBACK_COMPLETE` — it must be deleted and recreated from scratch.)

---

8. What to Review Next (highest-yield gaps this project didn't cover)

- **Blue/green deployments via CodeDeploy** (you only built rolling update) — know the `appspec.yaml` format and the two-target-group pattern

- **Auto Scaling for ECS services** — target tracking on CPU/memory, and Application Auto Scaling API
- **X-Ray tracing** — you have this flagged as a future project; it's a distinct troubleshooting-domain topic
- **Secrets Manager / Parameter Store integration** into task definitions (`secrets` block) — you used only env vars, but the exam tests the secrets-injection pattern specifically
- **VPC endpoints** (Interface endpoints for ECR API/DKR, S3 Gateway endpoint) — the "internet-free Fargate" pattern, directly relevant to the subnet issue you hit

---

Appendix: Command Reference You've Now Used in Anger

```bash
aws ecr get-login-password | docker login --username AWS --password-stdin ...
aws ecr describe-images --repository-name <repo> --query "imageDetails[].imageT
aws ecs register-task-definition --cli-input-json file://task-definition.json
aws ecs create-service --deployment-configuration "maximumPercent=200,minimumHe
aws ecs describe-services --query "services[0].deployments"
aws ecs describe-tasks --query "tasks[].stoppedReason"
aws cloudformation deploy --capabilities CAPABILITY_NAMED_IAM
aws cloudformation describe-stack-events --query "StackEvents[?contains(Resourc
aws codepipeline get-pipeline-state --name <pipeline>
aws logs tail /ecs/<app> --follow
```

Every one of these is fair game as a "what does this command do" or "what would you run to diagnose X" exam question.